

Replication of DLoc: Deep Learning Based Indoor Localization Using WiFi CSI

Abstract

This report presents our replication of DLoc, a deep learning framework for sub-meter indoor localization using WiFi Channel State Information (CSI), originally published at ACM MobiCom 2020 by Ayyalasomayajula et al. We replicate the Figure 10b experiment (Jacobs Hall, complex environment with 4 access points) using the publicly available WILD dataset. Training was performed on Kaggle using two NVIDIA T4 GPUs across multiple sessions. Our replication achieves a median localization error of **0.70 m** and 90th percentile error of **1.70 m**, within 9.4% and 6.3% of the paper’s reported 0.64 m and 1.60 m respectively. We identify and analyze seven sources of discrepancy, including a dataset composition deviation (3,805 extra training samples from unsplit time-instance-1 data), multi-session training artifacts (optimizer state, learning rate scheduler, and random seed resets), hardware and software differences, and reporting methodology.

1 Introduction

Indoor localization is a critical enabling technology for navigation, asset tracking, and context-aware services in GPS-denied environments. DLoc [1] proposes a deep learning approach that processes Angle-of-Arrival (AoA) feature images derived from WiFi CSI measurements to predict user locations on a 2D floor plan with sub-meter accuracy.

The DLoc architecture consists of a shared ResNet-based encoder and two decoder branches: a *location decoder* that predicts a 2D Gaussian heatmap over the floor plan, and a *consistency decoder* that reconstructs timing-offset-free AoA features to regularize the encoder representations. This multi-task design forces the encoder to learn features robust to hardware timing offsets present in commodity WiFi devices.

The goal of this work is to replicate the paper’s results on the complex indoor environment (Jacobs Hall, Figure 10b) using the publicly released WILD dataset and codebase, identify any discrepancies from the reported numbers, and provide a rigorous analysis of the sources of these discrepancies.

2 Dataset

2.1 Source

We use the WILD (Wireless Indoor Localization Dataset) released by the UCSD WCSNG lab alongside the DLoc paper. The dataset contains pre-computed AoA feature images generated from raw WiFi CSI channel matrices collected using a custom MIMO-OFDM testbed operating at 80 MHz bandwidth on the 5 GHz band.

2.2 Environment

The experiment corresponds to Figure 10b of the original paper: Jacobs Hall at UC San Diego, a complex indoor environment spanning approximately 1,500 square feet with 4 access points (APs). The spatial resolution is 0.1 m per pixel, yielding feature images of size 161×361 pixels ($16.1 \text{ m} \times 36.1 \text{ m}$).

2.3 Data Composition

The WILD dataset for Jacobs Hall consists of two raw feature files collected one hour apart in the same environment: `features_jacobs_Jul28.mat` (time instance 1, 11,440 samples) and `features_jacobs_Jul28_2.mat` (time instance 2, 14,199 samples). Pre-defined split indices in separate `.mat` files partition each time instance into FOV (field-of-view, line-of-sight to APs) and non-FOV (obstructed/NLOS) subsets, further divided into 70% train and 30% test within each time instance.

The paper states: “*we train on 80% data collected on two different time instances and test on the rest 20%.*” The intended setup uses only the *training split* of time instance 1 (7,635 samples) combined with the training split of time instance 2 (9,939 samples), yielding an 80/20 train-test ratio overall. Testing is performed exclusively on time instance 2 test splits (4,260 samples), evaluating temporal generalization.

Table 1 shows the files used in our replication.

Table 1: Dataset files used for the Figure 10b replication.

Split	File	Samples
Train	<code>dataset_jacobs_July28.mat</code>	11,440*
Train	<code>dataset_non_fov_train_jacobs_July28_2.mat</code>	9,351
Train	<code>dataset_fov_train_jacobs_July28_2.mat</code>	588
Test	<code>dataset_non_fov_test_jacobs_July28_2.mat</code>	4,008
Test	<code>dataset_fov_test_jacobs_July28_2.mat</code>	252

***Dataset composition issue.** Our `dataset_jacobs_July28.mat` contains all 11,440 samples from the raw feature file—the full, unsplit time instance 1. However, the data preparation script (`prepare_data.py`) would produce this file with only 7,635 samples by selecting `fov_train_idx` (632) + `non_fov_train_idx` (7,003) from the split index file. This means our training set includes 3,805 extra samples: 3,272 that the split indices designate as time-instance-1 “test” data, plus 533 samples outside any defined split. As a result, our training set contains 21,379 samples (83.4% train ratio) instead of the intended 17,574 samples (80.5% train ratio). The implications of this discrepancy are analyzed in Section 6.

The total dataset size on disk is approximately 37 GB in HDF5/MATLAB v7.3 format.

2.4 Data Format

Each `.mat` file contains three arrays:

- `features_w_offset` $\in \mathbb{R}^{N \times 4 \times 161 \times 361}$: AoA heatmaps *with* timing offsets. These serve as the network input.

- `features_wo_offset` $\in \mathbb{R}^{N \times 4 \times 161 \times 361}$: AoA heatmaps *without* timing offsets. These serve as the target for the consistency decoder.
- `labels_gaussian_2d` $\in \mathbb{R}^{N \times 161 \times 361}$: Ground-truth location encoded as a 2D Gaussian centered on the true position. These serve as the target for the location decoder.

Each sample has 4 channels corresponding to the 4 access points. The localization error is computed as the Euclidean distance between the `argmax` of the predicted heatmap and the `argmax` of the ground-truth heatmap, scaled by 0.1 m/pixel.

2.5 Data Loading Strategy

Due to the large dataset size (37 GB), we implemented a `LazyMultiHDF5Dataset` that reads samples on demand from HDF5 files rather than loading the entire dataset into RAM. Each `__getitem__` call performs three small HDF5 slice reads totaling approximately 700 KB per sample, resulting in near-zero RAM usage regardless of dataset size.

3 Model Architecture

The DLoc network follows an encoder–dual-decoder architecture with approximately 15 million parameters total.

3.1 Encoder

The shared encoder (`ResnetEncoder`) processes the 4-channel AoA input:

1. Conv2d 7×7 ($4 \rightarrow 4$ channels), InstanceNorm, Tanh
2. Conv2d 7×7 ($4 \rightarrow 64$ channels), InstanceNorm, ReLU
3. Conv2d 3×3 , stride 2 ($64 \rightarrow 128$ channels) — downsample $\times 2$
4. Conv2d 3×3 , stride 2 ($128 \rightarrow 256$ channels) — downsample $\times 2$
5. 6 ResNet blocks (256 channels each)

The encoder transforms the input from $[N, 4, 161, 361]$ to $[N, 256, 41, 91]$.

3.2 Location Decoder

The location decoder (`ResnetDecoder`, 9 total ResNet blocks, skipping the first 6) predicts a 2D location heatmap:

1. 3 ResNet blocks (256 channels)
2. ConvTranspose2d stride 2 ($256 \rightarrow 128$) — upsample $\times 2$
3. ConvTranspose2d stride 2 ($128 \rightarrow 64$) — upsample $\times 2$
4. Conv2d 7×7 ($64 \rightarrow 1$), Sigmoid

Output shape: $[N, 1, 161, 361]$. Loss: MSE + L1 regularization ($\lambda_{\text{reg}} = 5 \times 10^{-4}$).

3.3 Consistency (Offset) Decoder

The consistency decoder (`ResnetDecoder`, 12 total ResNet blocks, skipping the first 6) reconstructs the offset-free AoA features:

1. 6 ResNet blocks (256 channels)
2. ConvTranspose2d stride 2 ($256 \rightarrow 128$) — upsample $\times 2$
3. ConvTranspose2d stride 2 ($128 \rightarrow 64$) — upsample $\times 2$
4. Conv2d 7×7 ($64 \rightarrow 4$), Sigmoid

Output shape: $[N, 4, 161, 361]$. Loss: MSE ($\lambda_{\text{reg}} = 0$).

4 Training Configuration

4.1 Hardware

Training was performed on Kaggle using two NVIDIA Tesla T4 GPUs (16 GB VRAM each, 32 GB total) with `torch.nn.DataParallel` for multi-GPU parallelism. The original paper used 4 GPUs.

4.2 Hyperparameters

Table 2: Training hyperparameters.

Parameter	Value
Batch size	32
Total epochs	50
Optimizer	Adam ($\beta_1 = 0.5$, $\beta_2 = 0.999$)
Learning rate	1×10^{-5}
Weight decay	1×10^{-5}
LR scheduler	StepLR ($\gamma = 0.9$)
Encoder decay interval	50 epochs
Decoder decay interval	20 epochs
Offset decoder decay interval	50 epochs
Weight initialization	Xavier (gain = 0.02)
Normalization	Instance normalization
Random seed	0 (both PyTorch and NumPy)

4.3 Multi-Session Training

Due to Kaggle’s session time limits, training was conducted across multiple sessions:

- **Session 1:** Epochs 1–27
- **Session 2:** Epochs 28–39, resuming from epoch 27 checkpoint
- **Session 3:** Epochs 40–50, resuming from epoch 39 checkpoint

At each session boundary, model weights were saved and reloaded. However, the optimizer state and learning rate scheduler state were *not* preserved, introducing discrepancies from a single continuous training run (discussed in Section 6).

5 Results

5.1 Final Performance (Epoch 50)

Table 3: Localization error comparison: our replication vs. the original paper (Figure 10b, Jacobs Hall).

Metric	Our Result	Paper Result	Difference
Median error	0.70 m	0.64 m	+9.4%
90th percentile	1.70 m	1.60 m	+6.3%
99th percentile	3.78 m	3.20 m	+18.1%

5.2 Training Convergence

Table 4 shows the epoch-by-epoch test metrics during the final training session (epochs 40–50).

Table 4: Test localization error per epoch (final session).

Epoch	Median (m)	P90 (m)	P99 (m)
40	0.728	1.746	4.132
41	0.721	1.726	3.828
42	0.721	1.803	4.421
43	0.707	1.700	4.282
44	0.721	1.666	3.968
45	0.707	1.656	4.095
46	0.762	1.803	4.690
47	0.728	1.773	4.210
48	0.721	1.911	5.085
49	0.762	1.769	4.326
50	0.700	1.700	3.782

5.3 Training vs. Test Performance

At epoch 50, the training set metrics were: median = 0.14 m, P90 = 0.36 m, P99 = 1.89 m. The significant gap between training error (0.14 m) and test error (0.70 m) indicates the expected generalization behavior consistent with the paper—the model is evaluated on data from a different time instance than the training data.

5.4 Metric Oscillation

The test metrics exhibit oscillation rather than monotonic decrease in epochs 40–50 (e.g., median varies between 0.700 and 0.762 m). This is expected behavior caused by the discrete nature of the localization metric: the error is computed from `argmax` on integer pixel coordinates, so a prediction shifting by a single pixel (± 0.1 m) can change the median. The underlying MSE loss decreases smoothly, but the `argmax`-based evaluation metric is inherently quantized.

6 Analysis of Discrepancies

Our results are within 10% of the paper on median and P90, which constitutes a successful replication. However, we identify seven sources of discrepancy that collectively account for the remaining gap.

6.1 Source 1: Dataset Composition Deviation

Impact: Moderate (compensating). As detailed in Section 2.3, our training file `dataset_jacobs_July28.mat` contains all 11,440 samples from time instance 1 rather than the intended 7,635-sample training split. Table 5 quantifies the difference.

Table 5: Training set composition: our run vs. intended setup.

	Our Run	Intended (Paper)
Time instance 1 samples	11,440 (full file)	7,635 (train split)
Time instance 2 train	9,939	9,939
Total training	21,379	17,574
Total testing	4,260	4,260
Train ratio	83.4%	80.5%
Extra training samples	3,805 (+21.7%)	

The 3,805 extra samples include 3,272 samples that the split indices designate as time-instance-1 “test” data and 533 samples outside any defined split. While this is not data leakage in the strict sense—the test evaluation is performed exclusively on time instance 2—it means the model trains on a larger and differently composed dataset than the paper intended. More training data generally improves generalization, so this deviation likely *partially compensates* for the negative effects of the other discrepancy sources, making our results appear closer to the paper than they would be with the correct 7,635-sample split.

6.2 Source 2: Multi-Session Optimizer State Loss

Impact: High. The Adam optimizer maintains two per-parameter statistics: the first moment m_t (exponential moving average of gradients) and the second moment v_t (exponential moving average of squared gradients):

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \quad (1)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \quad (2)$$

$$\theta_t = \theta_{t-1} - \eta \frac{m_t / (1 - \beta_1^t)}{(\sqrt{v_t / (1 - \beta_2^t)}) + \epsilon} \quad (3)$$

With approximately 15 million parameters, Adam stores 30 million accumulated statistics. When training is resumed across Kaggle sessions, these statistics reset to zero, forcing Adam to re-estimate gradient moments from scratch. This causes temporarily misscaled update steps at the beginning of each new session, perturbing the optimization trajectory from the ideal single-run path.

6.3 Source 3: Learning Rate Scheduler Reset

Impact: Moderate. The decoder uses a `StepLR` scheduler with $\gamma = 0.9$ and step interval of 20 epochs. In a single 50-epoch run, the learning rate decays at epochs 20 and 40:

Table 6: Learning rate schedule: ideal vs. multi-session.

Epoch Range	Ideal Single Run	Our Multi-Session Run
1–19	1.0×10^{-5}	1.0×10^{-5}
20–27	0.9×10^{-5}	0.9×10^{-5}
28–39	0.9×10^{-5}	1.0×10^{-5} (scheduler reset)
40	0.81×10^{-5}	1.0×10^{-5} (scheduler reset)
41–47	0.81×10^{-5}	1.0×10^{-5}
48–50	0.81×10^{-5}	1.0×10^{-5}

In our multi-session run, the scheduler resets at each session boundary, so the second decay (at epoch 40) never occurs. The decoder trains at a higher learning rate than intended in later epochs, which can cause the oscillation observed in Table 4 and prevent the model from fine-tuning into a tighter optimum.

6.4 Source 4: Random Seed Reset Across Sessions

Impact: Moderate. The training script initializes random seeds at startup:

```
torch.manual_seed(0)
np.random.seed(0)
```

In a single continuous run, the random state evolves naturally across all 50 epochs, producing a unique data shuffle order per epoch. In our multi-session setup, the seed resets to 0 at the start of each Kaggle session. This means the `DataLoader` shuffle order for epoch 28 (start of session 2) is *identical* to epoch 1 (start of session 1), and epoch 40 (start of session 3) also repeats the same order. Different gradient sequences lead to a different weight trajectory compared to a single continuous run.

6.5 Source 5: Hardware and Parallelism Differences

Impact: Low–Moderate. The original paper used 4 GPUs, while our replication uses 2 NVIDIA T4 GPUs with `torch.nn.DataParallel`. `DataParallel` splits each batch across GPUs, computes forward and backward passes independently, then averages gradients on the primary GPU. With different GPU counts:

- The per-GPU micro-batch size differs (8 per GPU with 2 GPUs vs. 8 per GPU with 4 GPUs), affecting Instance Normalization statistics computed per-sample.
- Floating-point reduction order differs, introducing small numerical differences that compound over 50 epochs.
- T4 GPUs use a different microarchitecture (Turing) than the GPUs likely used in the original work, with different floating-point rounding behavior.

6.6 Source 6: Software Environment Differences

Impact: Low. Our replication uses PyTorch 2.x on CUDA 12.x, while the original paper (published 2020) likely used PyTorch 1.x on CUDA 10.x. Key differences include:

- cuDNN convolution algorithm selection: different algorithms can produce slightly different floating-point results.
- Default non-determinism in cuDNN: unless explicitly disabled via `torch.backends.cudnn.deterministic = True`, cuDNN selects the fastest convolution algorithm, which may vary between runs.
- Changes in PyTorch’s internal implementations of operations like Instance Normalization between major versions.

6.7 Source 7: Reporting Methodology

Impact: Unknown. Research papers typically report the best results from potentially multiple training runs. Our results represent a single training run. Additionally, the paper may report the best-epoch result (minimum median error across all epochs), while our epoch-by-epoch results show that individual epochs occasionally achieve lower errors (e.g., epoch 45: median = 0.707 m, P90 = 1.656 m).

6.8 Summary of Discrepancy Sources

Table 7: Summary of identified sources of discrepancy.

#	Source	Impact	Description
1	Dataset composition	Moderate [†]	3,805 extra training samples from unsplit time-instance-1 file; 83.4% vs. intended 80.5% train ratio
2	Adam state reset	High	Loss of 30M accumulated gradient statistics at each session boundary
3	LR scheduler reset	Moderate	Decoder misses second decay at epoch 40; trains at higher LR than intended
4	Random seed reset	Moderate	Data shuffle order repeats across sessions; different gradient sequence vs. continuous run
5	Hardware differences	Low–Mod	2 vs. 4 GPUs; different GPU architecture; different gradient averaging
6	Software versions	Low	PyTorch 2.x vs. 1.x; cuDNN non-determinism
7	Reporting methodology	Unknown	Paper may report best of multiple runs

[†]Compensating effect: more training data likely *improves* results, partially offsetting the negative effects of sources 2–4.

7 Corrective Measures

We identify three actionable fixes to eliminate the identified discrepancies in future runs.

7.1 Fix 1: Dataset Reprocessing

The `dataset_jacobs_July28.mat` file should be regenerated using `prepare_data.py`, which applies the split indices to produce a file containing only the 7,635 training-designated samples (`fov_train_idx + non_fov_train_idx`) rather than the full 11,440-sample raw file. This restores the paper’s intended 80/20 train-test ratio.

7.2 Fix 2: Optimizer and Scheduler State Persistence

We modified `modelADT.py` to save and restore optimizer and scheduler state alongside model weights. Specifically, each call to `save_networks(epoch)` now additionally saves:

- The Adam optimizer `state_dict` (first and second moment estimates for all parameters)
- The StepLR scheduler `state_dict` (internal step counter)

Upon resuming with `continue_train = True`, the `setup()` method loads these states after restoring model weights, ensuring the optimization trajectory is continuous across sessions.

7.3 Fix 3: Random Seed Continuity

The random seed reset can be addressed by saving the PyTorch and NumPy random states (`torch.random.get_rng_state()`, `np.random.get_state()`) alongside model checkpoints and restoring them on resume. Alternatively, performing a single uninterrupted training run eliminates this issue entirely.

7.4 Expected Impact

With fixes 1–3 applied, a multi-session run becomes mathematically equivalent to a single continuous run (up to floating-point non-determinism). However, since fix 1 *removes* 3,805 training samples, the net effect on final accuracy depends on whether the benefit of correct data composition outweighs the loss of additional training data. We expect the corrected setup to converge closer to the paper’s reported results, as it faithfully reproduces the intended experimental conditions.

8 Conclusion

We successfully replicate the DLoc localization system on the complex Jacobs Hall environment (Figure 10b), achieving a median error of 0.70 m and 90th percentile error of 1.70 m compared to the paper’s 0.64 m and 1.60 m. We identified seven sources of discrepancy through rigorous analysis. The primary sources are: (1) a dataset composition deviation where the time-instance-1 training file contained 3,805 extra samples beyond the intended split, (2) loss of Adam optimizer state across Kaggle session boundaries, (3) learning rate scheduler reset preventing the intended decay schedule, and (4) random

seed reset altering data shuffle order. Secondary contributions come from hardware differences (2 vs. 4 GPUs), software version differences (PyTorch 2.x vs. 1.x), and potential reporting methodology differences.

Notably, the dataset deviation (source 1) acts as a compensating factor—the additional training data likely *improved* our results, partially masking the degradation caused by the multi-session training artifacts (sources 2–4). The net result of these opposing effects is a 6–9% gap from the paper, which constitutes a successful replication within the expected variance for deep learning reproducibility.

These results validate the effectiveness of deep learning approaches for sub-meter indoor WiFi localization and demonstrate that the DLoc framework can be replicated on consumer-grade cloud GPU infrastructure with careful attention to dataset preparation and training state management.

References

- [1] R. Ayyalasomayajula, A. Arun, C. Wu, S. Sharma, A. R. Sethi, D. Vasisht, and D. Bharadia, “Deep Learning based Wireless Localization for Indoor Navigation,” in *Proc. ACM MobiCom*, 2020.